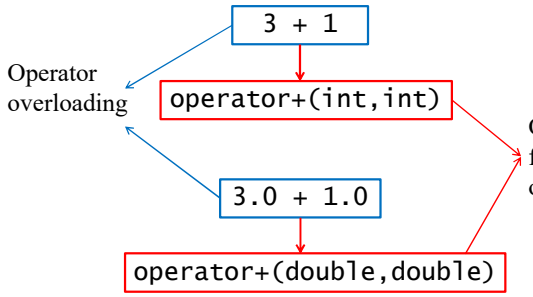
Chapter 10 Operator Overloading: String and Array Objects

OBJECTIVES

- What operator overloading is and how it makes programs more readable and programming more convenient. (运算符重载)
- To redefine (overload) operators to work with objects of user-defined classes.(重定义作用在用户自定义类对象上的运算符)
- The differences between overloading unary and binary operators.(重载一元和二元运算符的区别)
- To convert objects from one class to another class. (对象的类型转换)
- When to, and when not to, overload operators.(何时需要、合适不需要重载运算符)

10.1 Introduction(cont.)

- Operator overloading



10.1 Introduction(cont.)

- Operator overloading for user-defined objects
  - Generally, operator overloading is necessary for class object
  - Straightforward and natural way to extend C++
  - We will learn:
    - Fundamental concepts and restrictions
    - How to implement? Global *v.s.* Member function
    - Convert constructor
    - User-defined `Array` class, `String` class

Topics

10.2 Fundamentals of Operator Overloading

- To use an operator on a class object
  - It must be overloaded for that class
    - With three exceptions: (can also be overloaded)
      - Assignment operator (=)
      - Address operator (&)
      - Comma operator (,)
- Overloading provides concise notation

```
complex1.add( complex2 );   v.s.
complex1 + complex2;
```

10.2 Fundamentals of Operator Overloading (cont.)

- Operator function definition
  - Function name: `operator@`
    - Keyword: `operator`
    - Operand: @
  - Example:

```
Complex operator+(const Complex& a);
```

- Operator function invocation
  - Example:

```
complex1 + complex2;
```


Topics

10.3 Restrictions on Operator Overloading

Operators that can be overloaded

+	-	*	/	%	^	&	|
~	!	=	<	>	+=	-=	*=
/=	%=	^=	&=	|=	<<	>>	>>=
<<=	==	!=	<=	>=	&&	||	++
--	->*	,	->	[]	()	new	delete
new[]	delete[]						

(通过指针的)成员指针访问运算符

Operators that cannot be overloaded

.	.*	::	?:

(通过对象的)成员指针访问运算符　　条件运算符

➡ Appendix A

10.3 Restrictions on Operator Overloading (cont.)

- Cannot create new operators
- Cannot change
  - Precedence of operator (优先级)
  - Associativity (结合律)
  - Number of operands(操作数数目)

Unary operator: `+1, -1, !0, …`
Binary operator: `1+2, 1-2, 1*2, 1/2, …`
Ternary operator: `?:`

重载哪个运算符由操作数数目来决定

Unary/binary operator:

	+	-	*	&
Unary	positive	negative	pointer	address of
Binary	add	subtract	multiply	bitwise and

10.3 Restrictions on Operator Overloading (cont.)

- How an operator works on objects of fundamental types cannot be changed by operator overloading.
- Operator overloading works only with objects of user-defined types
  - `int + int` X
  - `complex + complex` √
  - `complex + int` √

Topics

10.4 Operator Functions as Class Members vs. Global Functions

- Non-`static` member function

```
class String {
public:
    bool operator!() const;
};
```

- Global function
  - `Friend` function (access non-`public` data)
  - Non-`friend` function (access `public` data)

```
class PhoneNumber {
  friend ostream & operator<< ( ostream &,
                    const PhoneNumber & );
};
```


10.4 Operator Functions as Class Members vs. Global Functions (cont.)

- Operator function as member function
  - Definition format:

Return type | Function name | Operands

```
T className :: operator op (…)
{
    ……  //function body
}
```


10.4 Operator Functions as Class Members vs. Global Functions (cont.)

- Operator function as member function(cont.)
  - Use `this*` to implicitly get left (or the only) operand
  - Left (or the only) operand must be an object (or object reference) of the same class as operator function
  - Operators `()`, `[]`, `->` or any assignment operator must be overloaded as member function

Number of parameters = Number of operands - 1

10.4 Operator Functions as Class Members vs. Global Functions (cont.)

- Operator function as member function (cont.)

Examples:

```
A  a1 ,a2 , a3;
a3  = a1 * a2;  ➡ a3 = a1.operator*(a2);
a3  = a1 ++;    ➡ a3 = a1.operator++();
a3  += a1;      ➡ a3.operator+=(a1);
```


10.4 Operator Functions as Class Members vs. Global Functions (cont.)

- Operator function as global function
  - Definition format:

Return type | Function name | Operands

```
T operator op (…)
{
    ……  //function body
}
```


10.4 Operator Functions as Class Members vs. Global Functions (cont.)

- Operator function as global function(cont.)
  - No `this*`
  - Need parameters for all operands
  - Can have left object of different class than operator or of fundamental type
  - Can be a `friend` to access non-`public` data

Number of parameters = Number of operands

10.4 Operator Functions as Class Members vs. Global Functions (cont.)

- Operator function as global function (cont.)

Examples:

```
A  a1 ,a2 , a3;
a3  = a1 * a2;  ➡  a3 = operator*(a1,a2);
a3  = a1 ++;    ➡  a3 = operator++(a1);
a3  += a1;      ➡  operator+=(a3,a1);
```


10.4 Operator Functions as Class Members vs. Global Functions (cont.)

- Commutative(可交换) operators
  - May want + to be commutative
    - So both "a + b" and "b + a" work
  - Suppose we have two different classes
    - ComplexClass + Int
    - Int + ComplexClass

Topics

10.5 Overloading Stream Insertion and Stream Extraction Operators

- Example
  - cin >> phone; // (123) 456-7890
  - cout << phone; // (123) 456-7890

```
13  class PhoneNumber
14  {
17  private:
18     string areaCode; // 3-digit area code
19     string exchange; // 3-digit exchange
20     string line; // 4-digit line
21  };
```



```
1  // Fig. 10.4: PhoneNumber.cpp
2  // Overloaded stream insertion and stream extraction operators
3  // for class PhoneNumber.
4  #include <iomanip>
5  using std::setw;
6
7  #include "PhoneNumber.h"
8
12 ostream &operator<<( ostream &output, const PhoneNumber &number )
13 {
14    output << "(" << number.areaCode << ") "
15       << number.exchange << "-" << number.line;
16    return output; // enables cout << a << b << c;
17 }
```

Allows `cout << phone;` interpreted as: `operator<<(cout,phone);`

Not recursive invoking. << for objects of `string` class is standard.

Return reference for cascading calls (e.g: `cout << phone1 << phone2`)

Allows `cin >> phone;` interpreted as: `operator>>(cin,phone);`

```
18
22 istream &operator>>( istream &input, PhoneNumber &number )
23 {
24    input.ignore(); // skip (
25    input >> setw( 3 ) >> number.areaCode;
26    input.ignore( 2 ); // skip ) and space
27    input >> setw( 3 ) >> number.exchange;
28    input.ignore(); // skip dash (-)
29    input >> setw( 4 ) >> number.line;
30    return input; // enables cin >> a >> b >> c;
31 }
```

`ignore` skips specified number of characters from input (1 by default)

When used with `cin` and `strings`, `setw(n)` restricts the number of characters read to `n`

Return reference for cascading calls (e.g: `cin >> phone1 >> phone2`)

(123) 456-7890

```
Enter phone number in the form (123) 456-7890:
(800) 555-1212
The phone number entered was: (800) 555-1212
```

Topics

10.6 Overloading Unary Operators

- Overloading unary operators

```
Object op      or      op Object
```


10.6 Overloading Unary Operators(cont.)

(1) Overloaded as non-`static` member function with no argument

- Be interpreted as:

```
Object.operator op()
```

操作数由对象`Object`通过`this`指针隐式传递

10.6 Overloading Unary Operators(cont.)

(2) Overload as global function with one argument

- Argument must be either an object of the class or a reference to an object of the class.
- Be interpreted as:

```
operator op (Object)
```

操作数由参数`Object`传递

Topics

10.7 Overloading Binary Operators

- Overloading binary operators

```
ObjectL op ObjectR
```

10.7 Overloading Binary Operators(cont.)

(1) Overload as non-`static` member function with one argument

- One of the arguments must be neither an object of the class or a reference to an object of the class
- Be interpreted as:

```
ObjectL.operator op( ObjectR )
```

左操作数由`ObjectL`通过`this`指针传递，
右操作数由参数`ObjectR`传递

10.7 Overloading Binary Operators(cont.)

(2) Overload as global function with two arguments

- Be interpreted as:

```
operator op( ObjectL, ObjectR )
```

左右操作数均由参数传递

Topics


```
1 // Fig. 10.10: Array.h
2 // Array class for storing arrays of integers
3 #ifndef ARRAY_H
4 #define ARRAY_H
5
6 #include <iostream>
7 using std::ostream;
8 using std::istream;
9
10 class Array
11 {
12    ...
34 private:
35    int size; // pointer-based array size
36    int *ptr; // pointer to first element of pointer-based array
37 };
38
39 #endif
```



```
1 // Fig. 10.10: Array.h
10 class Array
11 {
12    friend ostream &operator<<( ostream &, const Array & );
13    friend istream &operator>>( istream &, Array & );
14 public:
15    Array( int = 10 );          // default constructor
16    Array( const Array & );     // copy constructor
17    ~Array();                   // destructor
18    int getSize() const;        // return size
19
20    const Array &operator=( const Array & ); // assignment operator
21    bool operator==( const Array & ) const;  // equality operator
22
23    // inequality operator; returns opposite of == operator
24    bool operator!=( const Array &right ) const
25    {
26       return ! ( *this == right ); // invokes Array::operator==
27    }
28
29    // subscript operator for non-const objects returns lvalue
30    int &operator[]( int );
31
32    // subscript operator for const objects returns rvalue
33    int operator[]( int ) const;
34    ...
37 };
```



```
110 // Fig.10.11: Array.cpp
118 istream &operator>>( istream &input, Array &a )
119 {
120    for ( int i = 0; i < a.size; i++ )
121       input >> a.ptr[ i ];
123    return input; // enables cin >> x >> y;
124 }
125
127 ostream &operator<<( ostream &output, const Array &a )
128 {
129    int i;
130
131    // output private ptr-based array
132    for ( i = 0; i < a.size; i++ )
133    {
134       output << setw( 12 ) << a.ptr[ i ];
135
136       if ( ( i + 1 ) % 4 == 0 ) // 4 numbers per row of output
137       output << endl;
138    }
140    if ( i % 4 != 0 ) // end last line of output
141       output << endl;
142
143    return output; // enables cout << x << y;
144 }
```

```
26
29 Array::Array( const Array &arrayToCopy )
30    : size( arrayToCopy.size )
31 {
32    ptr = new int[ size ]; // create space for pointer-based array
33
34    for ( int i = 0; i < size; i++ )
35       ptr[ i ] = arrayToCopy.ptr[ i ]; // copy into object
36 }
37
39 Array::~Array()
40 {
41    delete [] ptr; // release pointer-based array space
42 }
43
45 int Array::getSize() const
46 {
47   return size; // number of elements in Array
48 }
49
```



```
50 // overloaded assignment operator;
51 // const return avoids: ( a1 = a2 ) = a3
52 const Array &Array::operator=( const Array &right )
53 {
54    if ( &right != this ) // avoid self-assignment
55    {
58       if ( size != right.size )
59       {
60          delete [] ptr; // release space
61          size = right.size; // resize this object
62          ptr = new int[ size ]; // create space for array copy
63       }
64
65       for ( int i = 0; i < size; i++ )
66          ptr[ i ] = right.ptr[ i ]; // copy array into object
67    }
68
69    return *this; // enables x = y = z, for example
70 }
```



```
72 // determine if two Arrays are equal and
73 // return true, otherwise return false
74 bool Array::operator==( const Array &right ) const
75 {
76    if ( size != right.size )
77       return false; // arrays of different number of elements
78
79    for ( int i = 0; i < size; i++ )
80       if ( ptr[ i ] != right.ptr[ i ] )
81          return false; // Array contents are not equal
82
83    return true; // Arrays are equal
84 }
```



```
88 int &Array::operator[]( int subscript )
89 {
90  // check for subscript out-of-range error
91  if ( subscript < 0 || subscript >= size )
92  {
93     cerr << "\nError: Subscript " << subscript
94        << " out of range" << endl;
95     exit( 1 ); // terminate program; subscript out of range
96  }
98  return ptr[ subscript ]; // reference return
99 }
100
101
103 int Array::operator[]( int subscript ) const
104 {
105    // check for subscript out-of-range error
106    if ( subscript < 0 || subscript >= size )
107    {
108       cerr << "\nError: Subscript " << subscript
109          << " out of range" << endl;
110       exit( 1 ); // terminate program; subscript out of range
111    }
113    return ptr[ subscript ]; // returns copy of this element
114 }
```



```
1  // Fig. 10.9: fig10_09.cpp
2  // Array class test program.
3  #include <iostream>
4  using std::cout;
5  using std::cin;
6  using std::endl;
7
8  #include "Array.h"
9
10 int main()
11 {
12    Array integers1( 7 ); // seven-element Array
13    Array integers2; // 10-element Array by default
14
15    // print integers1 size and contents
16    cout << "Size of Array integers1 is "
17       << integers1.getSize()
18       << "\nArray after initialization:\n" << integers1;
19
20    // print integers2 size and contents
21    cout << "\nSize of Array integers2 is "
22       << integers2.getSize()
23       << "\nArray after initialization:\n" << integers2;
```



```
25    // input and print integers1 and integers2
26    cout << "\nEnter 17 integers:" << endl;
27    cin >> integers1 >> integers2;
28
29    cout << "\nAfter input, the Arrays contain:\n"
30       << "integers1:\n" << integers1
31       << "integers2:\n" << integers2;
32
33    // use overloaded inequality (!=) operator
34    cout << "\nEvaluating: integers1 != integers2" << endl;
35
36   if ( integers1 != integers2 )
37   cout << "integers1 and integers2 are not equal" << endl;
```

```
39    // create Array integers3 using integers1 as an
40    // initializer; print size and contents
41    Array integers3( integers1 ); // invokes copy constructor
42
43    cout << "\nSize of Array integers3 is "
44      << integers3.getSize()
45      << "\nArray after initialization:\n" << integers3;
46
47    // use overloaded assignment (=) operator
48    cout << "\nAssigning integers2 to integers1:" << endl;
49    integers1 = integers2; // note target Array is smaller
50
51    cout << "integers1:\n" << integers1
52       << "integers2:\n" << integers2;
53
54    // use overloaded equality (==) operator
55    cout << "\nEvaluating: integers1 == integers2" << endl;
56
57    if ( integers1 == integers2 )
58       cout << "integers1 and integers2 are equal" << endl;
```



```
60    // use overloaded subscript operator to create rvalue
61    cout << "\nintegers1[5] is " << integers1[ 5 ];
62
63    // use overloaded subscript operator to create lvalue
64    cout << "\n\nAssigning 1000 to integers1[5]" << endl;
65    integers1[ 5 ] = 1000;
66    cout << "integers1:\n" << integers1;
67
68    // attempt to use out-of-range subscript
69    cout << "\nAttempt to assign 1000 to integers1[15]" << endl;
70    integers1[ 15 ] = 1000; // ERROR: out of range
71    return 0;
72 } // end main
```


Topics

10.9 Converting between Types(cont.)

- Conversions among user-define types
  - Conversion constructor (转换构造函数)
    - Other type → Class type
  - Conversion operator (转换运算符)
    - Class type → Other type

10.9 Converting between Types(cont.)

- Conversion constructor (转换构造函数)
  - Definition
    - Single-argument constructor
  - Objectiveness
    - Fundamental type → A particular class type
    - Another class type → A particular class type
  - Format

Target type　Source type

```
className( T ) {
   … //converting methods
}
```


10.9 Converting between Types(cont.)

- Conversion/cast operator (转换运算符)
  - Definition
    - A special member function
  - Objectiveness
    - A particular class type → Fundamental type
    - A particular class type → Another class type
  - Format

Format

Keyword　　Target type

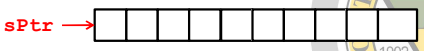
```
ClassName :: operator T ( ) {
   ... // converting methods
   return data_of_T;
}
```

1. Conversion operator function is a non-`static` member function
2. Type `T` can be fundamental or user-defined type
3、No parameter, no return type, but must return an object of type `T`

Topics

10.10 Case Study: `String` Class (cont.)

- class `String`
  - Use a dynamic array
  - `new[]`
  - `delete[]`
- Fig 10.1




```
10 class String
11 {
      … …
54 private:
55    int length; // string length (not counting null
terminator)
56    char *sPtr; // pointer to start of pointer-based
string
```



```
1  Fig. 10.1: String.h
10 class String
11 {
12   friend ostream &operator<<( ostream &, const String & );
13   friend istream &operator>>( istream &, String & );
14 public:
15   // conversion/default constructor
15   String( const char * = "" );
16   // copy constructor
16   String( const String & );
17   // destructor
17   ~String();
18   // assignment operator
19   const String &operator=( const String & );
20   // concatenation operator
20   const String &operator+=( const String & );
```

```
22   bool operator!() const; // is String empty?
23   bool operator==( const String & ) const; // test s1 == s2
24   bool operator<( const String & ) const; // test s1 < s2
25
26   // test s1 != s2
27   bool operator!=( const String &right ) const
28   {
29      return !( *this == right );
30   }
31
32   // test s1 > s2
33   bool operator>( const String &right ) const
34   {
35      return right < *this;
36   }
37
38   // test s1 <= s2
39   bool operator<=( const String &right ) const
40   {
41      return !( right < *this );
42   }
```

Code reusability


```
50   char &operator[]( int ); // subscript operator(lvalue)
51   char operator[]( int ) const; // subscript operator(rvalue)
52   // return a substring
52   String operator()( int, int = 0 ) const;
53   int getLength() const; // return string length
54 private:
55   int length;
56   char *sPtr;
57
58   void setString( const char * ); // utility function
59 };
```

```
21 // conversion (and default) constructor converts char * to String
22 String::String( const char *s )
23    : length( ( s != 0 ) ? strlen( s ) : 0 )
24 {
25    cout<<"Conversion (and default) constructor: " <<s<<endl;
26    setString( s ); // call utility function
27 }
```

```
159 void String::setString( const char *string2 )
160 {
161    sPtr = new char[ length + 1 ]; // allocate memory
162
163    if ( string2 != 0 ) // if string2 is not null pointer,
164       strcpy( sPtr, string2 ); // copy literal to object
165    else // if string2 is a null pointer
166       sPtr[ 0 ] = '\0'; // empty string
167 }
```



```
62 const String &String::operator+=( const String &right )
63 {
64   size_t newLength = length + right.length; // new length
65   char *tempPtr = new char[ newLength + 1 ]; // create memory
66
67   strcpy( tempPtr, sPtr ); // copy sPtr
68   strcpy( tempPtr + length, right.sPtr ); // copy right.sPtr
69
70   delete [] sPtr; // reclaim old space
71   sPtr = tempPtr; // assign new array to sPtr
72   length = newLength; // assign new length to length
73   return *this; // enables cascaded calls
74 }
```



```
49   cout << "The substring s1(0, 14), is:"
50      << s1( 0, 14 ) << "\n\n";
```

```
123 String String::operator()(int index,int subLength) const
124 {
140   char *tempPtr = new char[ len + 1 ];
143   strncpy( tempPtr, &sPtr[ index ], len );
144   tempPtr[ len ] = '\0';
145
146  //create temporary String object contain the substring
147   String tempString( tempPtr );
148   delete [] tempPtr; // delete temporary array
149   return tempString; // return copy of temporary String
150 }
```



```
49   cout << "The substring s1(0, 14), is:"
50      << s1( 0, 14 ) << "\n\n";
```

```
123 String String::operator()(int index,int subLength) const
124 {
140   char *tempPtr = new char[ len + 1 ];
143   strncpy( tempPtr, &sPtr[ index ], len );
144   tempPtr[ len ] = '\0';
145
146  //create temporary String object contain the substring
147   String tempString( tempPtr );
148   delete [] tempPtr; // delete temporary array
149   return tempString; // return copy of temporary String
150 }
```



```
88 // Is this String less than right String?
89 bool String::operator<( const String &right ) const
90 {
91    return strcmp( sPtr, right.sPtr ) < 0;
92 } // end function operator<
93
94 // return reference to character in String as a modifiable lvalue
95 char &String::operator[]( int subscript )
96 {
97    // test for subscript out of range
98    if ( subscript < 0 || subscript >= length )
99    {
100      cerr << "Error: Subscript " << subscript
101         << " out of range" << endl;
102      exit( 1 ); // terminate program
103   } // end if
104
105   return sPtr[ subscript ]; // non-const return; modifiable lvalue
106} // end function operator[]
107
108// return reference to character in String as rvalue
109char String::operator[]( int subscript ) const
110{
111   // test for subscript out of range
112   if ( subscript < 0 || subscript >= length )
113   {
114      cerr << "Error: Subscript " << subscript
115         << " out of range" << endl;
116      exit( 1 ); // terminate program
117   } // end if
118
119   return sPtr[ subscript ]; // returns copy of this element
120} // end function operator[]
121
```

```
76 // is this String empty?
77 bool String::operator!() const
78 {
79    return length == 0;
80 } // end function operator!
81
82 // is this String equal to right String?
83 bool String::operator==( const String &right ) const
84 {
85    return strcmp( sPtr, right.sPtr ) == 0;
86 } // end function operator==
87
152// return string length
153int String::getLength() const
154{
155   return length;
156} // end function getLength
157
158// utility function called by constructors and operator=
159void String::setString( const char *string2 )
160{
161   sPtr = new char[ length + 1 ]; // allocate memory
162
163   if ( string2 != 0 ) // if string2 is not null pointer, copy contents
164      strcpy( sPtr, string2 ); // copy literal to object
165   else // if string2 is a null pointer, make this an empty string
166      sPtr[ 0 ] = '\0'; // empty string
167} // end function setString
168
```

```
175
176// overloaded input operator
177istream &operator>>( istream &input, String &s )
178{
179   char temp[ 100 ]; // buffer to store input
180   input >> setw( 100 ) >> temp;
181   s = temp; // use String class assignment operator
182   return input; // enables cascading
183} // end function operator>>
```



```
1  // Fig. 11.11: fig11_11.cpp
2  // String class test program.
3  #include <iostream>
4  using std::cout;
5  using std::endl;
6  using std::boolalpha;
7
8  #include "String.h"
9
10 int main()
11 {
12    String s1( "happy" );
13    String s2( " birthday" );
14    String s3;
15
16    // test overloaded equality and relational operators
17    cout << "s1 is \"" << s1 << "\"; s2 is \"" << s2
18       << "\"; s3 is \"" << s3 << '\"'
19       << boolalpha << "\n\nThe results of comparing s2 and s1:"
20       << "\ns2 == s1 yields " << ( s2 == s1 )
21       << "\ns2 != s1 yields " << ( s2 != s1 )
22       << "\ns2 >  s1 yields " << ( s2 > s1 )
23       << "\ns2 <  s1 yields " << ( s2 < s1 )
24       << "\ns2 >= s1 yields " << ( s2 >= s1 )
25       << "\ns2 <= s1 yields " << ( s2 <= s1 );
26
27
28    // test overloaded String empty (!) operator
29    cout << "\n\nTesting !s3:" << endl;
```

```
31    if ( !s3 )
32    {
33       cout << "s3 is empty; assigning s1 to s3;" << endl;
34       s3 = s1; // test overloaded assignment
35       cout << "s3 is \"" << s3 << "\"";
36    } // end if
37
38    // test overloaded String concatenation operator
39    cout << "\n\ns1 += s2 yields s1 = ";
40    s1 += s2; // test overloaded concatenation
41    cout << s1;
42
43    // test conversion constructor
44    cout << "\n\ns1 += \" to you\" yields" << endl;
45    s1 += " to you"; // test conversion constructor
46    cout << "s1 = " << s1 << "\n\n";
47
48    // test overloaded function call operator () for substring
49    cout << "The substring of s1 starting at\n"
50       << "location 0 for 14 characters, s1(0, 14), is:\n"
51       << s1( 0, 14 ) << "\n\n";
52
53    // test substring "to-end-of-String" option
54    cout << "The substring of s1 starting at\n"
55       << "location 15, s1(15), is: "
56       << s1( 15 ) << "\n\n";
57
58    // test copy constructor
59    String *s4Ptr = new String( s1 );
60    cout << "\n*s4Ptr = " << *s4Ptr << "\n\n";
```

```
61
62    // test assignment (=) operator with self-assignment
63    cout << "assigning *s4Ptr to *s4Ptr" << endl;
64    *s4Ptr = *s4Ptr; // test overloaded assignment
65    cout << "*s4Ptr = " << *s4Ptr << endl;
66
67    // test destructor
68    delete s4Ptr;
69
70    // test using subscript operator to create a modifiable lvalue
71    s1[ 0 ] = 'H';
72    s1[ 6 ] = 'B';
73    cout << "\ns1 after s1[0] = 'H' and s1[6] = 'B' is: "
74       << s1 << "\n\n";
75
76    // test subscript out of range
77    cout << "Attempt to assign 'd' to s1[30] yields:" << endl;
78    s1[ 30 ] = 'd'; // ERROR: subscript out of range
79    return 0;
80 } // end main
```


Topics

10.13 Standard Library Class `string`

- C++ standard class `string`
- `#include <string>`
- §21 详细介绍`string`类

Topics


```
1  // Fig. 10.6: Date.h
9  class Date
10 {
11    friend ostream &operator<<( ostream &, const Date & );
12 public:
13    Date( int m = 1, int d = 1, int y = 1900 );// constructor
14    void setDate( int, int, int ); // set month, day, year
15    Date &operator++(); // prefix increment operator
16    Date operator++( int ); // postfix increment operator
17    const Date &operator+=( int ); // add days, modify object
18    bool leapYear( int ) const; // is date in a leap year?
19    bool endOfMonth( int ) const; //is date at the end of month?
20 private:
21    int month;
22    int day;
23    int year;
25    static const int days[]; // array of days per month
26    void helpIncrement(); // utility function
27 };
```



```
1  // Fig. 10.7: Date.cpp
7  const int Date::days[] =
8     { 0, 31, 28, 31, 30, 31, 30, 31, 31, 30, 31, 30, 31 };
9
10 // Date constructor
11 Date::Date( int m, int d, int y )
12 {
13    setDate( m, d, y );
14 }
15
16 // set month, day and year
17 void Date::setDate( int mm, int dd, int yy )
18 {
19    month = ( mm >= 1 && mm <= 12 ) ? mm : 1;
20    year = ( yy >= 1900 && yy <= 2100 ) ? yy : 1900;
21
23    if ( month == 2 && leapYear( year ) ) // test for leap year
24       day = ( dd >= 1 && dd <= 29 ) ? dd : 1;
25    else
26       day = ( dd >= 1 && dd <= days[ month ] ) ? dd : 1;
27 }
```



```
56 //if the year is a leap year,return true; otherwise,return false
57 bool Date::leapYear( int testYear ) const
58 {
59    if ( testYear % 400 == 0 ||
60       ( testYear % 100 != 0 && testYear % 4 == 0 ) )
61       return true; // a leap year
62    else
63       return false; // not a leap year
64 }
```



```
29 // overloaded prefix increment operator
30 Date &Date::operator++()
31 {
32    helpIncrement(); // increment date
33    return *this; // reference return to create an lvalue
34 }
35
36 // overloaded postfix increment operator; note that the
37 // dummy integer parameter does not have a parameter name
38 Date Date::operator++( int )
39 {
40    Date temp = *this; // hold current state of object
41    helpIncrement();
42
43    // return unincremented, saved, temporary object
44    return temp; // value return; not a reference return
45 }
```



```
75 // function to help increment the date
76 void Date::helpIncrement()
77 {
78    // day is not end of month
79    if ( !endOfMonth( day ) )
80       day++; // increment day
81    else
82       if ( month < 12 ) // day is end of month and month < 12
83       {
84          month++; // increment month
85          day = 1; // first day of new month
86       }
87       else // last day of year
88       {
89          year++; // increment year
90          month = 1; // first month of new year
91          day = 1; // first day of new month
92       }
93 }
```

```
66 // determine whether the day is the last day of the month
67 bool Date::endOfMonth( int testDay ) const
68 {
69    if ( month == 2 && leapYear( year ) )
70       return testDay == 29; // last day of Feb. in leap year
71    else
72       return testDay == days[ month ];
73 }
```

```
47 // add specified number of days to date
48 const Date &Date::operator+=( int additionalDays )
49 {
50    for ( int i = 0; i < additionalDays; i++ )
51       helpIncrement();
52
53    return *this; // enables cascading
54 }
```



```
95 // overloaded output operator
96 ostream &operator<<( ostream &output, const Date &d )
97 {
98    static char *monthName[ 13 ] = { "", "January", "February",
99       "March", "April", "May", "June", "July", "August",
100       "September", "October", "November", "December" };
101    output << monthName[ d.month ] << ' ' << d.day << ", " <<
       d.year;
102    return output; // enables cascading
103 }
```



```
1  // Fig. 10.8: fig10_08.cpp
2  // Date class test program.
3  #include <iostream>
4  #include "Date.h" // Date class definition
5  using namespace std;
6
7  int main()
8  {
9     Date d1( 12, 27, 2010 ); // December 27, 2010
10    Date d2; // defaults to January 1, 1900
11
12    cout << "d1 is " << d1 << "\nd2 is " << d2;
13    cout << "\n\nd1 += 7 is " << ( d1 += 7 );
14
15    d2.setDate( 2, 28, 2008 );
16    cout << "\n\n  d2 is " << d2;
17    cout << "\n++d2 is " << ++d2 << " (leap year allows 29th)";
18
19    Date d3( 7, 13, 2010 );
20
21    cout << "\n\nTesting the prefix increment operator:\n"
22       << "  d3 is " << d3 << endl;
23    cout << "++d3 is " << ++d3 << endl;
24    cout << "  d3 is " << d3;
```

Fig. 10.8 | Date class test program. (Part 1 of 2.)

```
25
26    cout << "\n\nTesting the postfix increment operator:\n"
27       << "  d3 is " << d3 << endl;
28    cout << "d3++ is " << d3++ << endl;
29    cout << "  d3 is " << d3 << endl;
30 } // end main
```

```
d1 is December 27, 2010
d2 is January 1, 1900

d1 += 7 is January 3, 2011

  d2 is February 28, 2008
++d2 is February 29, 2008 (leap year allows 29th)

Testing the prefix increment operator:
  d3 is July 13, 2010
++d3 is July 14, 2010
  d3 is July 14, 2010

Testing the postfix increment operator:
  d3 is July 14, 2010
d3++ is July 14, 2010
  d3 is July 15, 2010
```

Fig. 10.8 | Date class test program. (Part 2 of 2.)

Topics


```
1  // Fig. 10.12: Array.h
2  // Array class for storing arrays of integers.
3  #ifndef ARRAY_H
4  #define ARRAY_H
5
6  #include <iostream>
7  using std::ostream;
8  using std::istream;
9
10 class Array
11 {
12    friend ostream &operator<<( ostream &, const Array & );
13    friend istream &operator>>( istream &, Array & );
14 public:
15    explicit Array( int = 10 ); // default constructor
16    Array( const Array & ); // copy constructor
17    ~Array(); // destructor
18    int getSize() const; // return size
19
20    const Array &operator=( const Array & ); // assignment operator
21    bool operator==( const Array & ) const; // equality operator
```

```
1  // Fig. 10.13: Fig10_13.cpp
2  // Driver for simple class Array.
7  #include "Array.h"
8
9  void outputArray( const Array & ); // prototype
10
11 int main()
12 {
13    Array integers1( 7 ); // 7-element array
14    outputArray( integers1 ); // output Array integers1
15    outputArray( 3 ); // convert 3 to an Array and output Array's contents
16    outputArray( Array( 3 ) ); // explicit single-argument constructor call
17    return 0;
18 }
19
20 // print arra
21 void outputArray( const Array &arrayToOutput )
22 {
23    cout << "The Array received has " << arrayToOutput.getSize()
24       << " elements. The contents are:\n" << arrayToOutput << endl;
25 }
```

```
c:\books\2012\cpphtp9\examples\ch10\fig10_13\fig10_13.cpp(13): error C2664:
'outputArray' : cannot convert parameter 1 from 'int' to 'const Array &'
          Reason: cannot convert from 'int' to 'const Array'
          Constructor for class 'Array' is declared 'explicit'
```


Summary

- What operators can be overloaded? When? How? Restriction?
- Member Function v.s. Global Function
- Overloading Unary and Binary operators
- Conversion Constructor v.s. Conversion Operator
- `Array`, `String`, `Date` classes